

Data 260 Homework 1

Agentic AI, Docker, and AWS ECS Deployment

- **Part 0: Student Information and Configuration**

Name: Vrishin Dharmesh Kunnatham Parambath

SID4: 8844

PORT_BASE: 8744

PREFIX: s8844 SEED: 8844

VERIFY_SEED: 268844

DOMAIN_ID: 4

Assigned Domain: Open-source package vulnerabilities

Python Version: 3.12

Local Model: qwen3:8b

Hardware: Processor: Intel Core i7-10750H CPU @ 2.60 GHz

Memory: 16 GB RAM

Graphics: NVIDIA GeForce GTX 1650

Repository:

<https://github.com/Viki2183/data260-8844>

Evidence Commit: 60a8e21

Submission Tag: hw1

- **Part 1: HTML, JavaScript, Docker and AWS**

- 1) **Domain Schema**

My assigned domain is open-source package vulnerabilities. Before creating the form, I defined the following fields in DOMAIN_SCHEMA.md:

1. Package name: the name of the affected open-source package.
2. Vulnerability ID: a CVE number or another security advisory identifier.
3. Submitter email: the email address of the person submitting the report.
4. Vulnerability description: a detailed explanation of the vulnerability and its possible impact.
5. Severity category: Critical, High, Medium, or Low.
6. Terms accepted: records whether the submitter agreed to the terms and conditions.
7. Submission date: the date and time added by JavaScript after a successful submission.

2) HTML Implementation

I created an HTML page titled:

HW1-Vrishin Dharmesh Kunnatham Parambath

The page uses an <h1> heading named “Open-Source Package Vulnerability.”

The form contains:

1. A required package-name field with autofocus.
2. A required vulnerability-ID field.
3. A required email field.
4. A required vulnerability-description textarea.
5. A severity dropdown containing Critical, High, Medium, and Low.
6. A terms-and-conditions checkbox.
7. A “Submit Vulnerability Report” button.
8. A script reference to app.js at the end of the page.

```
9. <!DOCTYPE html>
10. <html lang="en">
11. <head>
12.     <meta charset="UTF-8">
13.     <meta name="viewport" content="width=device-width, initial-
    scale=1.0">
14.
15.     <title>HW1-Vrishin Dharmesh Kunnatham Parambath</title>
16.
17.     <style>
18.         body {
19.             max-width: 700px;
20.             margin: 40px auto;
21.             padding: 20px;
22.             font-family: Arial, sans-serif;
23.             background-color: #f4f6f8;
24.         }
25.
```

```
26.     h1 {
27.         color: #263238;
28.         text-align: center;
29.     }
30.
31.     form {
32.         padding: 25px;
33.         background-color: white;
34.         border-radius: 8px;
35.         box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
36.     }
37.
38.     label {
39.         display: block;
40.         margin-top: 15px;
41.         margin-bottom: 5px;
42.         font-weight: bold;
43.     }
44.
45.     input[type="text"],
46.     input[type="email"],
47.     textarea,
48.     select {
49.         width: 100%;
50.         padding: 10px;
51.         border: 1px solid #aaa;
52.         border-radius: 4px;
53.         box-sizing: border-box;
54.     }
55.
56.     textarea {
57.         min-height: 120px;
58.         resize: vertical;
59.     }
60.
61.     .terms {
62.         display: flex;
63.         align-items: center;
64.         gap: 8px;
65.         margin-top: 20px;
66.     }
67.
68.     .terms label {
69.         margin: 0;
70.         font-weight: normal;
71.     }
72.
73.     button {
```

```
74.         width: 100%;
75.         margin-top: 20px;
76.         padding: 12px;
77.         color: white;
78.         background-color: #c62828;
79.         border: none;
80.         border-radius: 4px;
81.         font-size: 16px;
82.         cursor: pointer;
83.     }
84.
85.     button:hover {
86.         background-color: #8e0000;
87.     }
88. </style>
89.</head>
90.
91.<body>
92.    <h1>Open-Source Package Vulnerability</h1>
93.
94.    <form id="vulnerabilityForm">
95.        <label for="packageName">Package Name</label>
96.        <input
97.            type="text"
98.            id="packageName"
99.            name="packageName"
100.            placeholder="Enter the affected package name"
101.            required
102.            autofocus
103.        >
104.
105.        <label for="vulnerabilityId">Vulnerability ID</label>
106.        <input
107.            type="text"
108.            id="vulnerabilityId"
109.            name="vulnerabilityId"
110.            placeholder="Enter a CVE or security advisory
            identifier"
111.            required
112.        >
113.
114.        <label for="submitterEmail">Submitter Email</label>
115.        <input
116.            type="email"
117.            id="submitterEmail"
118.            name="submitterEmail"
119.            placeholder="Enter your SJSU email address"
120.            required
```

```

121.         >
122.
123.         <label for="vulnerabilityDescription">
124.             Vulnerability Description
125.         </label>
126.         <textarea
127.             id="vulnerabilityDescription"
128.             name="vulnerabilityDescription"
129.             placeholder="Describe the vulnerability and its
potential impact"
130.             required
131.         ></textarea>
132.
133.         <label for="severity">Severity Category</label>
134.         <select id="severity" name="severity" required>
135.             <option value="" disabled selected>Select
severity</option>
136.             <option value="Critical">Critical</option>
137.             <option value="High">High</option>
138.             <option value="Medium">Medium</option>
139.             <option value="Low">Low</option>
140.         </select>
141.
142.         <div class="terms">
143.             <input
144.                 type="checkbox"
145.                 id="termsAccepted"
146.                 name="termsAccepted"
147.             >
148.             <label for="termsAccepted">
149.                 I agree to the terms and conditions.
150.             </label>
151.         </div>
152.
153.         <button type="submit">Submit Vulnerability
Report</button>
154.     </form>
155.
156.     <script src="app.js"></script>
157. </body>
158. </html>

```

3) JavaScript Implementation

I used an arrow function named `validateForm` to validate the form.

The function checks that:

1. The vulnerability description contains more than 25 characters.
2. The terms-and-conditions checkbox is selected.

If either validation fails, the application displays an alert explaining the problem.

After a successful submission, the program:

1. Converts the form data into a JSON string.
2. Parses the JSON string back into a JavaScript object.
3. Uses object destructuring to extract the package name and submitter email.
4. Uses the spread operator to add the current submission date and time.
5. Uses a closure to count successful form submissions.
6. Logs the JSON data, extracted fields, updated object, and submission count in the browser console.

```
"use strict";
```

```
// Closure that remembers the number of successful submissions.
```

```
const submissionCounter = (() => {
```

```
  let count = 0;
```

```
  return () => {
```

```
    count += 1;
```

```
    return count;
```

```
  };
```

```
})();
```

// Arrow function used to validate the description and terms checkbox.

```
const validateForm = () => {  
    const description = document  
        .getElementById("vulnerabilityDescription")  
        .value  
        .trim();  
  
    const termsAccepted =  
        document.getElementById("termsAccepted").checked;  
  
    if (description.length <= 25) {  
        alert(  
            "The vulnerability description must contain more than 25  
characters."  
        );  
        return false;  
    }  
  
    if (!termsAccepted) {  
        alert(  
            "You must agree to the terms and conditions before  
submitting."  
        );  
        return false;  
    }  
}
```

```
    return true;
};
```

```
document
```

```
    .getElementById("vulnerabilityForm")
    .addEventListener("submit", (event) => {
        event.preventDefault();
```

```
        if (!validateForm()) {
            return;
        }
```

```
        const formData = {
            packageName: document
                .getElementById("packageName")
                .value
                .trim(),
```

```
            vulnerabilityId: document
                .getElementById("vulnerabilityId")
                .value
                .trim(),
```

```
            submitterEmail: document
```

```
.getElementById("submitterEmail")  
.value  
.trim(),
```

```
vulnerabilityDescription: document  
.getElementById("vulnerabilityDescription")  
.value  
.trim(),
```

```
severity: document.getElementById("severity").value,
```

```
termsAccepted:  
    document.getElementById("termsAccepted").checked  
};
```

```
// Convert the form object into a JSON string.  
const jsonString = JSON.stringify(formData);
```

```
console.log("JSON string:");  
console.log(jsonString);
```

```
// Convert the JSON string back into a JavaScript object.  
const parsedObject = JSON.parse(jsonString);
```

```
// Extract the primary field and email using object
destructuring.

const { packageName, submitterEmail } = parsedObject;

console.log("Package name:", packageName);
console.log("Submitter email:", submitterEmail);

// Add the current date and time using the spread operator.
const updatedObject = {
    ...parsedObject,
    submissionDate: new Date().toISOString()
};

console.log("Updated object:");
console.log(updatedObject);

// Increase and display the closure-based submission count.
const submissionCount = submissionCounter();

console.log("Successful submission count:",
submissionCount);

alert("Vulnerability report submitted successfully!");
});
```

4) Local Docker Deployment

I created a Docker image using Nginx on Alpine Linux. I built the image from the repository root with:

```
docker build -f code/Dockerfile -t data260-8844-hw1 .
```

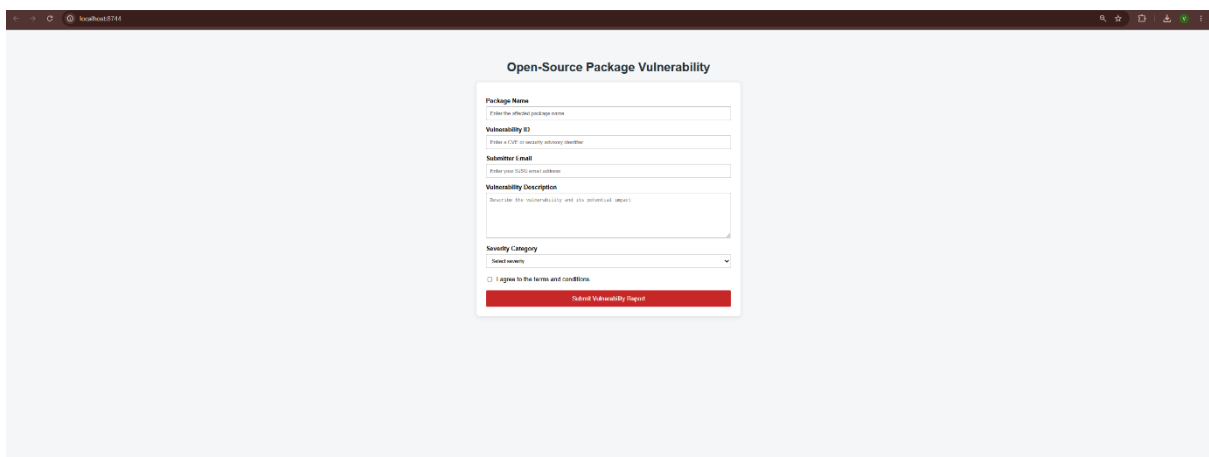
I ran the container with:

```
docker run -d -p 8744:80 --name data260-8844-container data260-8844-hw1
```

The application was available locally at:

<http://localhost:8744>

The page loaded correctly and displayed the complete open-source package vulnerability form.



The screenshot shows a web browser window with the address bar displaying 'localhost:8744'. The main content area features a form titled 'Open-Source Package Vulnerability'. The form includes the following fields and elements:

- Package Name:** A text input field with a placeholder 'Enter the affected package name'.
- Vulnerability ID:** A text input field with a placeholder 'Enter a CVE or security advisory identifier'.
- Submitter Email:** A text input field with a placeholder 'Enter your full email address'.
- Vulnerability Description:** A large text area with a placeholder 'Describe the vulnerability and its potential impact'.
- Severity Category:** A dropdown menu with 'Select severity' as the selected option.
- Agreement:** A checkbox labeled 'I agree to the terms and conditions'.
- Submit Button:** A red button labeled 'Submit Vulnerability Report'.

5) AWS ECS Deployment

I pushed the Docker image to Amazon Elastic Container Registry and deployed it using Amazon ECS with AWS Fargate.

I used the following AWS resources:

1. Region: us-east-1
2. ECR repository: s8844-hw1
3. ECS cluster: s8844-hw1-cluster
4. Task definition: s8844-hw1-task:1
5. ECS service: s8844-hw1-service

6. Security group: s8844-hw1-sg
7. Container port: 80
8. Desired tasks: 1
9. CPU: 0.25 vCPU
10. Memory: 0.5 GB
11. Public IP: enabled

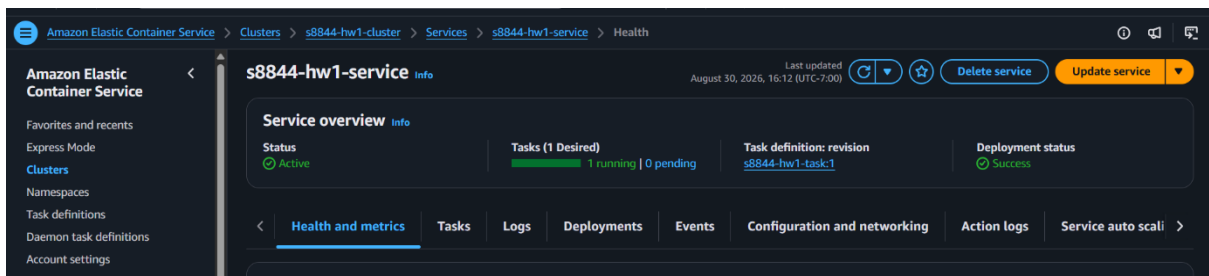
During the first deployment attempt, the ECS task could not retrieve authorization from Amazon ECR. The ECS event showed that the request to ECR on port 443 timed out. I inspected the security group and found that outbound traffic was incorrectly limited to port 80.

I corrected the outbound rule to allow HTTPS traffic on TCP port 443. After that, the task successfully downloaded the image from ECR.

The application was still initially unreachable from the browser because the security group had no inbound rules. I added an inbound HTTP rule allowing TCP port 80 from 0.0.0.0/0.

After these corrections, the ECS deployment completed successfully with:

- Desired tasks: 1
- Running tasks: 1
- Pending tasks: 0
- Failed tasks: 0



The application then loaded successfully using the ECS task's public IPv4 address.

The screenshot shows a web browser window with the address bar displaying "32.192.194.199". The page title is "Open-Source Package Vulnerability". The form contains the following fields:

- Package Name:** A text input field with the placeholder "Enter the affected package name".
- Vulnerability ID:** A text input field with the placeholder "Enter a CVE or security advisory identifier".
- Submitter Email:** A text input field with the placeholder "Enter your SJSU email address".
- Vulnerability Description:** A large text area with the placeholder "Describe the vulnerability and its potential impact".
- Severity Category:** A dropdown menu with "Selected severity" as the current selection.
- ☐ I agree to the terms and conditions.
- Submit Vulnerability Report:** A red button at the bottom of the form.

I submitted a valid vulnerability report and received the message “Vulnerability report submitted successfully!”

This screenshot shows the same form as the previous one, but with a success message overlay. The message box is dark with white text and an "OK" button. The form fields are now filled with the following data:

- Package Name:** requests
- Vulnerability ID:** CVE-2026-8844
- Submitter Email:** vrishindharmesh.kunnathamparambath@sjsu.edu
- Vulnerability Description:** An authentication bypass allows unauthorized remote users to access protected application resources and retrieve sensitive data without valid credentials.
- Severity Category:** Critical
- ☒ I agree to the terms and conditions.
- Submit Vulnerability Report:** A red button at the bottom of the form.

After collecting the required evidence, I changed the service’s desired task count to zero to stop continuing Fargate and public-IP usage.

• Part 2: Agentic AI

1) Exact Command Used

I ran the Planner, Reviewer, and Finalizer pipeline with the following command:

```
python code\agents_demo.py `
  --title "Critical Authentication Bypass in Requests Package" `
  --content "An authentication bypass allows unauthorized remote users to
access protected resources and retrieve sensitive application data." `
  --email "vrishindharmesh.kunnathamparambath@sjsu.edu" `
```

```
--model "qwen3:8b" `
--temperature 0.0 `
--strict
```

2) Explanation of the Pipeline

Step 1: Planner

The Planner reads the supplied title and content. It proposes exactly three topical tags and a summary containing no more than 25 words.

The tags are derived from the provided input. They are not hardcoded for my assigned domain.

Step 2: Reviewer

- The Reviewer checks whether:
- There are exactly three tags.
- Every tag is related to the supplied input.
- The summary is no longer than 25 words.
- Any detected problems need to be corrected.

Step 3: Finalizer

The Finalizer uses the Planner and Reviewer outputs to produce the final strict JSON result. The final result contains exactly three tags, one short summary, and an empty issues list.

Step 4: Publish Package

The publish package combines the original input, Planner and Reviewer transcript, final tags and summary, model name, temperature, timestamp, and latency.

➤ Planner and Reviewer Implementation

```
planner, planner_latency = call_agent(
    client=client,
    name="Planner",
    system_prompt=(
        "You are the Planner. Propose exactly three distinct "
        "topical tags and a summary of at most 25 words."
    ),
    task=task,
    title=args.title,
    content=args.content,
```

```

        context=[],
        strict=args.strict,
    )

    transcript.append({
        "agent": "Planner",
        "output": planner,
        "latency_ms": round(planner_latency, 2),
    })

    reviewer, reviewer_latency = call_agent(
        client=client,
        name="Reviewer",
        system_prompt=(
            "You are the Reviewer. Check whether the Planner's "
            "tags come from the input, whether there are exactly "
            "three tags, and whether the summary has at most "
            "25 words. Correct problems and list detected issues."
        ),
        task=task,
        title=args.title,
        content=args.content,
        context=transcript,
        strict=args.strict,
    )

    transcript.append({
        "agent": "Reviewer",
        "output": reviewer,
        "latency_ms": round(reviewer_latency, 2),
    })

```

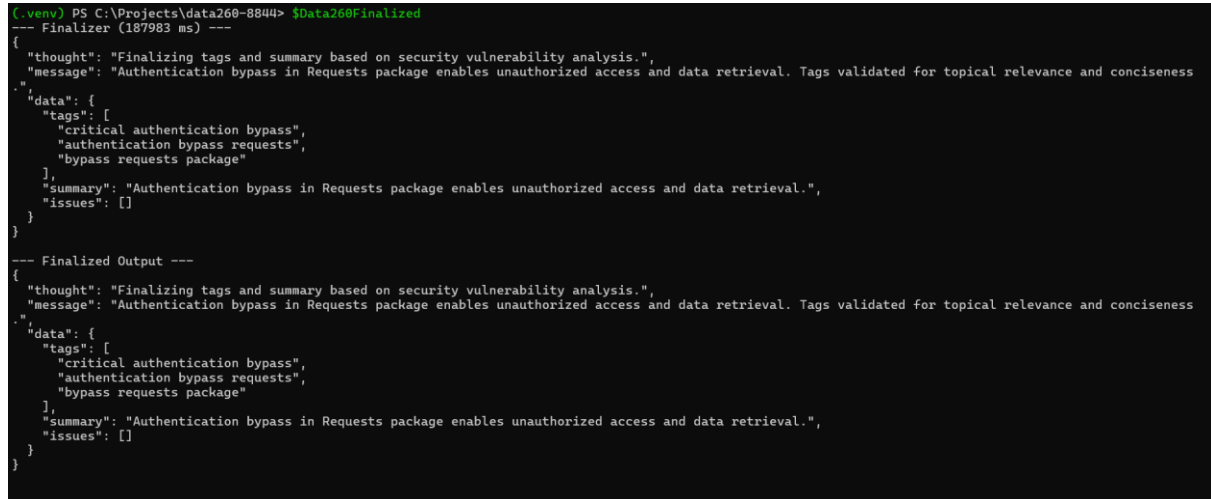
```
Windows PowerShell
(.venv) PS C:\Projects\data260-8844> Write-Output ""
(.venv) PS C:\Projects\data260-8844> Write-Output "DATA 260 HW1 - PLANNER AND REVIEWER OUTPUT"
DATA 260 HW1 - PLANNER AND REVIEWER OUTPUT
(.venv) PS C:\Projects\data260-8844> Write-Output ""
(.venv) PS C:\Projects\data260-8844> $Data260PlannerReviewer
--- Planner (198615 ms) ---
{
  "thought": "Identify security vulnerability aspects and impacted components.",
  "message": "Authentication bypass in Requests package enables unauthorized access and data retrieval.",
  "data": {
    "tags": [
      "critical authentication bypass",
      "authentication bypass requests",
      "bypass requests package"
    ],
    "summary": "Authentication bypass in Requests package enables unauthorized access and data retrieval.",
    "issues": []
  }
}
Turn token usage: input=697, output=41, total=738
--- Reviewer (198526 ms) ---
{
  "thought": "",
  "message": "Tags and summary prepared.",
  "data": {
    "tags": [
      "critical authentication bypass",
      "authentication bypass requests",
      "bypass requests package"
    ],
    "summary": "Authentication bypass in Requests package enables unauthorized access and data retrieval.",
    "issues": []
  }
}
Turn token usage: input=725, output=85, total=810
(.venv) PS C:\Projects\data260-8844> |
```

➤ Finalizer Implementation:

```
finalizer, finalizer_latency = call_agent(
    client=client,
    name="Finalizer",
    system_prompt=(
        "Perform the finalization step using the Planner and "
        "Reviewer outputs. Return exactly three tags, a summary "
        "of at most 25 words, and an empty issues array."
    ),
    task=task,
    title=args.title,
    content=args.content,
    context=transcript,
    strict=args.strict,
)
```

```
finalizer["data"]["issues"] = []
```

```
print("\n--- Finalized Output ---")
print(json.dumps(finalizer, indent=2))
```



```
(.venv) PS C:\Projects\data260-8844> $Data260Finalized
--- Finalizer (187983 ms) ---
{
  "thought": "Finalizing tags and summary based on security vulnerability analysis.",
  "message": "Authentication bypass in Requests package enables unauthorized access and data retrieval. Tags validated for topical relevance and conciseness",
  "data": {
    "tags": [
      "critical authentication bypass",
      "authentication bypass requests",
      "bypass requests package"
    ],
    "summary": "Authentication bypass in Requests package enables unauthorized access and data retrieval.",
    "issues": []
  }
}

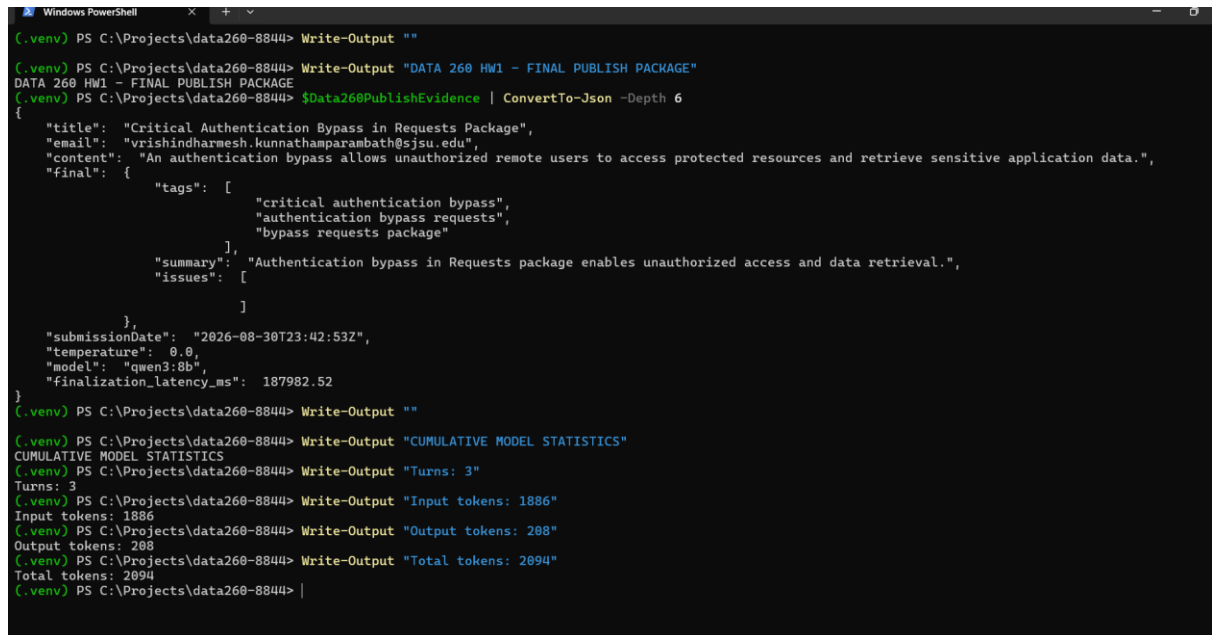
--- Finalized Output ---
{
  "thought": "Finalizing tags and summary based on security vulnerability analysis.",
  "message": "Authentication bypass in Requests package enables unauthorized access and data retrieval. Tags validated for topical relevance and conciseness",
  "data": {
    "tags": [
      "critical authentication bypass",
      "authentication bypass requests",
      "bypass requests package"
    ],
    "summary": "Authentication bypass in Requests package enables unauthorized access and data retrieval.",
    "issues": []
  }
}
```

➤ Final Publish-Package Construction

```
publish_package = {
    "title": args.title,
    "email": args.email,
    "content": args.content,
    "agents": {
        "transcript": transcript,
        "final": finalizer["data"],
    },
    "submissionDate": time.strftime(
        "%Y-%m-%dT%H:%M:%SZ",
        time.gmtime(),
    ),
    "temperature": args.temperature,
    "model": args.model,
    "finalization_latency_ms": round(
        finalizer_latency,
        2,
    ),
}
```

```
}
```

```
print("\n--- Publish Package ---")  
print(json.dumps(publish_package, indent=2))
```



```
(.venv) PS C:\Projects\data260-8844> Write-Output ""  
(.venv) PS C:\Projects\data260-8844> Write-Output "DATA 260 HW1 - FINAL PUBLISH PACKAGE"  
DATA 260 HW1 - FINAL PUBLISH PACKAGE  
(.venv) PS C:\Projects\data260-8844> $Data260PublishEvidence | ConvertTo-Json -Depth 6  
{  
  "title": "Critical Authentication Bypass in Requests Package",  
  "email": "vrishindharmesh.kunnathamparambath@sjsu.edu",  
  "content": "An authentication bypass allows unauthorized remote users to access protected resources and retrieve sensitive application data.",  
  "final": {  
    "tags": [  
      "critical authentication bypass",  
      "authentication bypass requests",  
      "bypass requests package"  
    ],  
    "summary": "Authentication bypass in Requests package enables unauthorized access and data retrieval.",  
    "issues": [  
    ]  
  },  
  "submissionDate": "2026-08-30T23:42:53Z",  
  "temperature": 0.0,  
  "model": "qwen3:8b",  
  "finalization_latency_ms": 187982.52  
}  
(.venv) PS C:\Projects\data260-8844> Write-Output ""  
(.venv) PS C:\Projects\data260-8844> Write-Output "CUMULATIVE MODEL STATISTICS"  
CUMULATIVE MODEL STATISTICS  
(.venv) PS C:\Projects\data260-8844> Write-Output "Turns: 3"  
Turns: 3  
(.venv) PS C:\Projects\data260-8844> Write-Output "Input tokens: 1886"  
Input tokens: 1886  
(.venv) PS C:\Projects\data260-8844> Write-Output "Output tokens: 208"  
Output tokens: 208  
(.venv) PS C:\Projects\data260-8844> Write-Output "Total tokens: 2094"  
Total tokens: 2094  
(.venv) PS C:\Projects\data260-8844> |
```

3) Required Short Answers

Q1. What three final tags did you obtain?

critical authentication bypass
authentication bypass requests
bypass requests package

Q2. What was the final summary?

“Authentication bypass in Requests package enables unauthorized access and data retrieval.”

The summary contains nine words, so it satisfies the maximum limit of 25 words.

Q3. Did the Reviewer change anything?

No. The Planner already produced exactly three input-related tags and a summary shorter than 25 words. Therefore, the Reviewer did not identify any issues or make corrections.

- **Part 3: Measuring Non-Determinism**

- 1) **Fixed Input**

I saved one unchanged input in:

reports/hw01/cases/nondeterminism_input.json

The fixed title was:

“Critical Authentication Bypass in Requests Package”

The fixed content was:

“An authentication bypass allows unauthorized remote users to access protected resources and retrieve sensitive application data.”

I ran the pipeline:

1. 20 times at temperature 0.7
2. 20 times at temperature 0.0
3. 40 times in total

The results were saved after every completed run so the experiment could resume safely if it was interrupted.

➤ Forty-Run Experiment Configuration

```
TEMPERATURES = [0.7, 0.0]
```

```
RUNS_PER_TEMPERATURE = 20
```

```
for temperature in TEMPERATURES:
```

```
    for run_number in range(
```

```
        1,
```

```
        RUNS_PER_TEMPERATURE + 1,
```

```
    ):
```

```
        key = (temperature, run_number)
```

```
        if key in completed:
```

```
            print(
```

```
                f"Skipping completed run: "
```

```
                f"temperature={temperature}, "
```

```
                f"run={run_number}"
```

```
            )
```

```
            Continue
```

2) Tag Variation Results

Metric	Temperature 0.7	Temperature 0.0
Distinct tag sets	7	3
Tags appearing in all 20 runs	critical authentication bypass	critical authentication bypass
Tags appearing in exactly one run	authentication bypass requests package	unauthorized access

3) Latency Results

Metric	Temperature 0.7	Temperature 0.0
p50 latency	955,924.94 ms	599,091.85 ms
p95 latency	1,224,709.49 ms	1,005,474.73 ms
p99 latency	1,418,972.12 ms	1,047,240.56 ms

```

Created C:\Projects\data260-8844\reports\hw01\verification.json
(.venv) PS C:\Projects\data260-8844> Get-Content reports\hw01\raw\nondeterminism_metrics.json
{
  "input_file": "reports\\hw01\\raw\\nondeterminism_results.json",
  "total_runs": 40,
  "temperature_0.7": {
    "temperature": 0.7,
    "run_count": 20,
    "distinct_tag_sets": 7,
    "tags_in_all_20_runs": [
      "critical authentication bypass"
    ],
    "tags_in_exactly_one_run": [
      "authentication bypass requests package"
    ],
    "latency_p50_ms": 955924.94,
    "latency_p95_ms": 1224709.49,
    "latency_p99_ms": 1418972.12
  },
  "temperature_0.0": {
    "temperature": 0.0,
    "run_count": 20,
    "distinct_tag_sets": 3,
    "tags_in_all_20_runs": [
      "critical authentication bypass"
    ],
    "tags_in_exactly_one_run": [
      "unauthorized access"
    ],
    "latency_p50_ms": 599091.85,
    "latency_p95_ms": 1005474.73,
    "latency_p99_ms": 1047240.56
  }
}

```

Figure: Nondeterminism experiment results for 20 runs at temperature 0.7 and 20 runs at temperature 0.0

4) Interpretation

Two users submitting the same title and content may receive different but still relevant tag phrases. The higher temperature produced more variation: temperature 0.7 produced seven distinct tag sets, while temperature 0.0 produced three.

The higher-temperature runs were also slower at the median and tail latency measurements.

Variation is acceptable when generating optional discovery tags because multiple relevant tag phrases can help users find the same vulnerability report.

Variation is not acceptable when assigning a vulnerability ID or severity level. Inconsistent IDs or severity values could cause incorrect tracking, prioritization, or remediation decisions.

The raw experiment results are stored in:

1. reports/hw01/raw/nondeterminism_results.json

2. reports/hw01/raw/nondeterminism_results.csv
3. reports/hw01/raw/nondeterminism_metrics.json
4. reports/hw01/RUN_LOG.txt
5. reports/hw01/METRICS.md

➤ Saving Results After Every Run

```
result = {  
    "temperature": temperature,  
    "run": run_number,  
    "tags": finalized["data"]["tags"],  
    "summary": finalized["data"]["summary"],  
    "latency_ms": round(latency_ms, 2),  
    **token_stats,  
    "timestamp": time.strftime(  
        "%Y-%m-%dT%H:%M:%SZ",  
        time.gmtime(),  
    ),  
}  
  
results.append(result)  
  
save_results(results)
```

Each completed result is appended and immediately saved. Therefore, the experiment can resume without losing previously completed runs if execution is interrupted.

- **Part 4: Model Client and Token Accounting**

- 1) Reusable Model Adapter**

- I created `src/model_client.py` as a reusable adapter for model calls. All model interactions go through this module.

- The adapter accepts a list of messages and provides a stable completion interface. It also records input tokens, output tokens, total tokens, turn count, and cumulative statistics.

- I created `code/hw1_client.py` as the command-line demonstration program.

- 2) AGENT.md Instructions**

- My AGENT.md file asks the model to perform strict bullet-only code review. The client checks every nonempty response line and reports whether the bullet-only format passed.

- 3) Five-Turn Token Statistics**

- After turn 3:

- 1. Turn count: 3
 2. Cumulative input tokens: 722
 3. Cumulative output tokens: 1,412
 4. Cumulative total tokens: 2,134
 5. Serialized history length: 969

- After turn 5:

- 1. Turn count: 5
 2. Cumulative input tokens: 1,544
 3. Cumulative output tokens: 2,109
 4. Cumulative total tokens: 3,653
 5. Serialized history length: 1,910

➤ Model Completion Interface and Token Accounting

```
def complete(
    self,
    messages: list[dict[str, str]],
    tools: list[Any] | None = None,
) -> dict[str, Any]:
    """Send messages to Ollama through one stable interface."""

    model = self.model

    if tools:
        model = model.bind_tools(tools)

    response = model.invoke(messages)

    usage = response.usage_metadata or {}
    metadata = response.response_metadata or {}

    input_tokens = int(
        usage.get(
            "input_tokens",
            metadata.get("prompt_eval_count", 0),
        )
    )

    output_tokens = int(
        usage.get(
            "output_tokens",
            metadata.get("eval_count", 0),
        )
    )

    total_tokens = int(
        usage.get(
            "total_tokens",
            input_tokens + output_tokens,
        )
    )

    self.turn_count += 1
```

```

self.cumulative_input_tokens += input_tokens
self.cumulative_output_tokens += output_tokens

token_usage = {
    "input_tokens": input_tokens,
    "output_tokens": output_tokens,
    "total_tokens": total_tokens,
}

return {
    "content": str(response.content),
    "usage": token_usage,
}

```

The reusable model adapter sends the conversation to Ollama and records the input, output, and total token usage for every model call.

➤ Cumulative Statistics

```

def stats(self, history: list[dict[str, str]]) -> dict[str, int]:
    """Return statistics without changing conversation history."""

    return {
        "turn_count": self.turn_count,
        "cumulative_input_tokens":
            self.cumulative_input_tokens,
        "cumulative_output_tokens":
            self.cumulative_output_tokens,
        "cumulative_total_tokens":
            self.cumulative_input_tokens
            + self.cumulative_output_tokens,
        "serialized_history_length":
            len(str(history)),
    }

```

The stats method reports cumulative token usage, completed model turns, and the serialized size of the conversation history without modifying that history.

➤ Five-Turn History, /stats, and Bullet Verification

```
history: list[dict[str, str]] = []
stats_snapshots: list[dict] = []

if user_input == "/stats":
    stats = client.stats(history)
    stats["after_turn"] = client.turn_count
    stats_snapshots.append(stats)

    print("\n/stats")
    print(json.dumps(stats, indent=2))
    print()

    save_conversation(
        history,
        stats_snapshots,
        client,
    )
    Continue

messages = [
    {
        "role": "system",
        "content": system_prompt,
    },
    *history,
]

result = client.complete(messages)
response = result["content"].strip()

print(f"\nAssistant:\n{response}")
print(
    "\nBullet-only verification:",
```

```

    "PASS" if is_bullet_only(response) else "FAIL",
    "\n",
)

```

```

history.append({
    "role": "assistant",
    "content": response,
})

```

The client preserves previous user and assistant messages in history. The `/stats` command records statistics after turns 3 and 5, while the verification check confirms that every non-empty response line uses bullet formatting.



```

Windows PowerShell
(.venv) PS C:\Projects\data260-8844> Write-Output ""
(.venv) PS C:\Projects\data260-8844> Write-Output "DATA 260 HW1 - MODEL CLIENT /STATS EVIDENCE"
DATA 260 HW1 - MODEL CLIENT /STATS EVIDENCE
(.venv) PS C:\Projects\data260-8844> Write-Output ""
(.venv) PS C:\Projects\data260-8844> $Data260ClientEvidence.stats_snapshots |
>> ConvertTo-Json -Depth 5
[
  {
    "turn_count": 3,
    "cumulative_input_tokens": 722,
    "cumulative_output_tokens": 1412,
    "cumulative_total_tokens": 2134,
    "serialized_history_length": 969,
    "after_turn": 3
  },
  {
    "turn_count": 5,
    "cumulative_input_tokens": 1544,
    "cumulative_output_tokens": 2109,
    "cumulative_total_tokens": 3653,
    "serialized_history_length": 1910,
    "after_turn": 5
  }
]
(.venv) PS C:\Projects\data260-8844> Write-Output ""
(.venv) PS C:\Projects\data260-8844> Write-Output "FINAL FIVE-TURN STATISTICS"
FINAL FIVE-TURN STATISTICS
(.venv) PS C:\Projects\data260-8844> $Data260ClientEvidence.final_stats |
>> ConvertTo-Json -Depth 5
{
  "turn_count": 5,
  "cumulative_input_tokens": 1544,
  "cumulative_output_tokens": 2109,
  "cumulative_total_tokens": 3653,
  "serialized_history_length": 1910
}

```

4) Why is prior conversation context resent with every turn?

Separate model requests are stateless. The model does not automatically remember earlier API calls. The client must resend the relevant conversation history so the model can understand the current request in context.

5) How is a system prompt different from a user message?

A system prompt defines the model's overall role, behavior, and constraints. In this project, it tells the model to perform strict bullet-only code review.

A user message contains the specific code or request that the model should process during an individual turn.

6) Why do input tokens grow during a conversation?

Input tokens grow because each new request includes more previous conversation history. As the conversation becomes longer, the serialized prompt sent to the model also becomes longer.

7) What eventually limits that growth?

The model's context-window size is the main limit. Available memory, processing time, latency, and computational cost also create practical limits.

- **Verification**

I ran:

`python code\verify_hw01.py`

The verification result showed:

`"all_passed": true`

The following checks passed:

1. Required files
2. HTML requirements
3. JavaScript requirements
4. Forty nondeterminism runs
5. Agent-output schema
6. Client-stat snapshots
7. Python syntax
8. Docker image existence

The verification result is saved in:

`reports/hw01/verification.json`

```

{
  "student": "Vrishin Dharmesh Kunnatham Parambath",
  "sid4": 8844,
  "port_base": 8744,
  "prefix": "s8844",
  "verify_seed": 268844,
  "all_passed": true,
  "checks": [
    {
      "name": "required_files",
      "passed": true,
      "details": "All required files are present."
    },
    {
      "name": "html_requirements",
      "passed": true,
      "details": "HTML requirements are present."
    },
    {
      "name": "javascript_requirements",
      "passed": true,
      "details": "JavaScript requirements are present."
    },
    {
      "name": "nondeterminism_runs",
      "passed": true,
      "details": "Total=40, temperature_0.7=20, temperature_0.0=20"
    },
    {
      "name": "agent_output_schema",
      "passed": true,
      "details": "All runs contain three tags and summaries of at most 25 words."
    },
    {
      "name": "client_stats_snapshots",
      "passed": true,
      "details": "Recorded /stats after turns: [3, 5]"
    },
    {
      "name": "python_syntax",
      "passed": true,
      "details": "All Python files compiled successfully."
    },
    {
      "name": "docker_image",
      "passed": true,
      "details": "Docker image data260-8844-hw1 exists."
    }
  ]
}

```

```

Created C:\Projects\data260-8844\reports\hw01\verification.json
(.venv) PS C:\Projects\data260-8844>

```

- **AI Use Disclosure**

1) What did I use an AI assistant for, and what did I do myself?

I used an AI assistant to help me understand the assignment, organize the repository, create starter code, troubleshoot Docker and AWS, and identify tests I needed to run. I personally created and ran the project files, tested the HTML and JavaScript validation, built the Docker image, ran the Ollama experiments, reviewed the generated results, deployed the application to AWS, and captured the required evidence.

2) What AI-produced output was wrong or unsuitable?

During an early code-review test, the model incorrectly claimed that:
`def divide(a, b): return a / b`
divided a by b twice.

3) How did I detect the problem?

I manually inspected the function and counted the division operators. The expression contains only one division operator, so the model's statement was incorrect.

4) What did I change, and why does it work now?

I updated AGENT.md to tell the model to review only operations that are visibly present in the supplied code and to avoid unsupported claims. I also required every response line to use bullet-only formatting and added `/no_think` to the system instructions used by the client. I restarted the official five-turn test and verified that the recorded responses followed the required bullet-only format. I still manually reviewed the responses because correct formatting does not guarantee factual correctness.

• Submission Information

Repository:

<https://github.com/Viki2183/data260-8844>

The following GitHub users were added as collaborators:

1. Sbnikitha
2. supriyaselvanganesan

The completed repository will be tagged:

hw1

The final uploaded PDF must be identical to:

reports/hw01/report.pdf
at the tagged commit.